

FY-26: Career Preview Program project Review (CPP-3)

Updated June 2026

Vellore Institute of Technology, Chennai

Project Details

PROJECT

QueryCraft — HPE Nonstop System Performance Report Generator using NLP Queries

UNIVERSITY

Vellore Institute of Technology, Chennai

CATEGORY

AI / Data Analytics · HPE Nonstop

github.com/Surjithk73/HPE_49

PROJECT PARTICIPANTS

 Santo Paul M

 Surjith Khannan

 Pragya Sekar

 Vishal Dev Verma

 Rudra Saxena

MENTORS

 Raghavendra MK

 Kaviraj Doddaiiah

 Rohan Emmanuel

 Dr. Viswanathan V

QueryCraft

Natural Language to SQL Performance Report Generator

Democratizing HPE Nonstop System Insights - ask in plain English, get instant, secure, exportable analytics.

NL → SQL

GPT-OSS 20B · LOCAL

POSTGRESQL · CHROMADB

PROJECT	CATEGORY	PLATFORM
QueryCraft	AI / Data Analytics	HPE Nonstop



Problem Statement

Why Querying Nonstop Data Hurts



The Challenge

Querying complex, historical HPE Nonstop server performance data requires deep SQL expertise and intricate knowledge of the underlying database schema.



The Bottleneck

Data analysts spend significant time writing complex, multi-table `JOIN` queries just to extract standard metrics.



The Need

A system that allows users to ask for performance insights in plain English and receive instant, accurate, and easily exportable data visualizations.

```
normalized_cpu_busy_pct.sql

-- Normalized CPU busy percentage per CPU
SELECT cpu_num,
       CAST(SUM(cpu_busy_time) * 100.0
            / NULLIF(SUM(delta_time * ipus),
                     0) AS DECIMAL(10, 4))
       AS normalized_busy_pct
FROM %db%.cpu
WHERE ipus > 0
GROUP BY cpu_num
ORDER BY normalized_busy_pct DESC
LIMIT 10000;
```



WHAT THE ANALYST ACTUALLY WANTS

“Show me the **CPU busy percentage** for each CPU in a **normalized way**.”

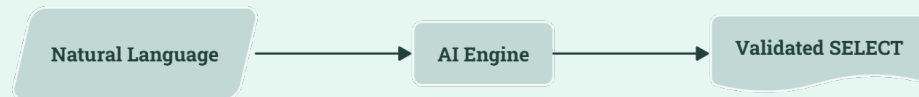
One sentence. QueryCraft generates, validates, and runs the SQL on the left — automatically.

Solution Proposal

QueryCraft: Plain English In. Secure SQL Out.

THE CONCEPT

QueryCraft converts plain English into context-aware, validated, read-only PostgreSQL queries — then auto-renders results as charts or downloadable reports.



CORE CAPABILITIES

LLM Integration



Converts plain English to exact, context-aware SQL statements using GPT-OSS 20B — an open-source model running locally, with no external API calls.

Automated Insights



Instantly auto-detects result structures to render data as Tables, Line Charts, or Bar Charts.

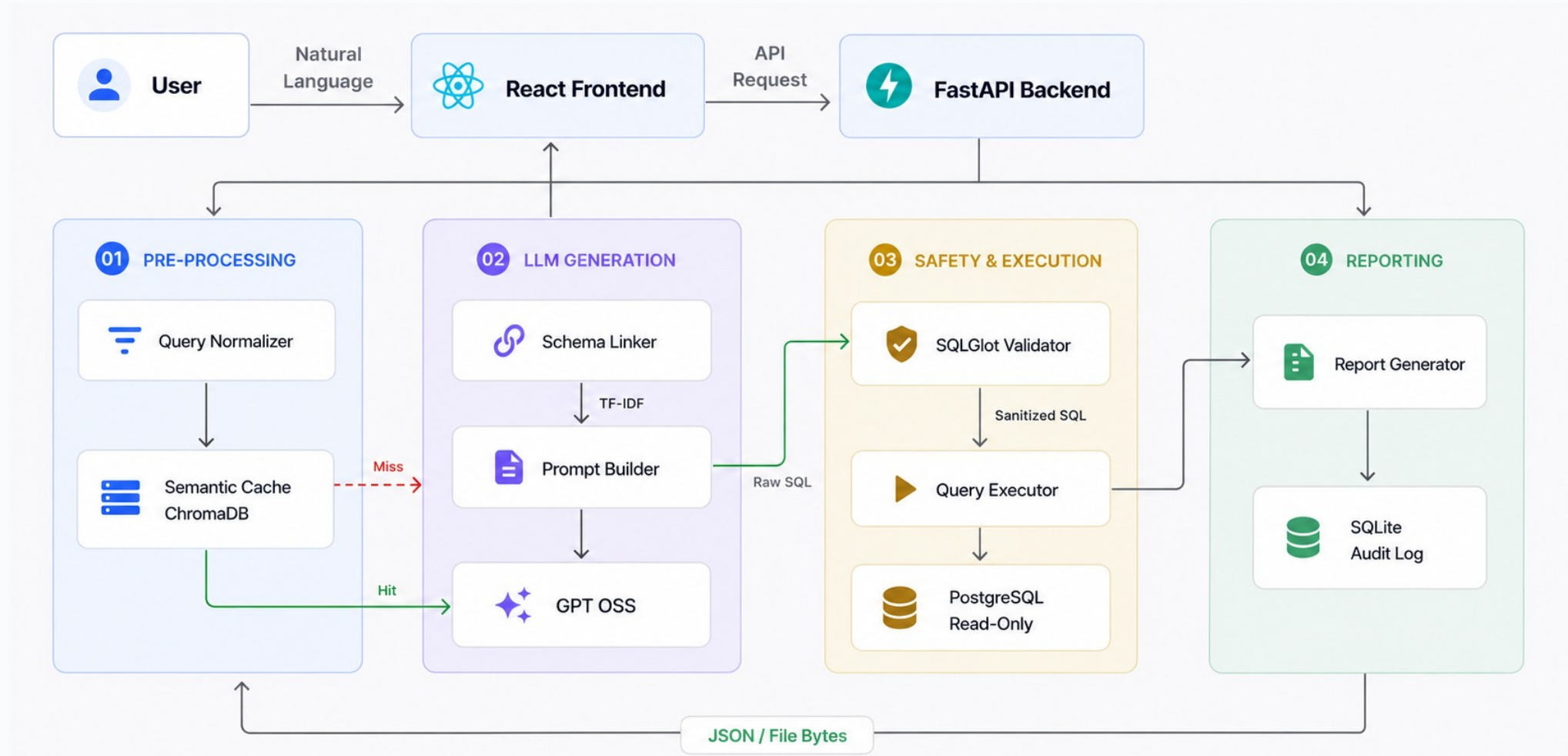
Zero-Friction Exporting



Built-in ability to download results instantly as CSV, Excel, or formatted PDF reports.

Architecture Diagram

End-to-End Query Pipeline



Input & Pre-Processing

Standardize the question, then short-circuit repeated queries with a semantic cache.

01 PRE-PROCESSING >

02 LLM GENERATION >

03 SAFETY & EXECUTION >

04 REPORTING



COMPONENT 01

Query Normalizer

Standardizes the user's plain English, corrects HPE-specific abbreviations (e.g., "proc" to "process"), and categorizes the domain (CPU, Disc, File).

CPU

Disc

File



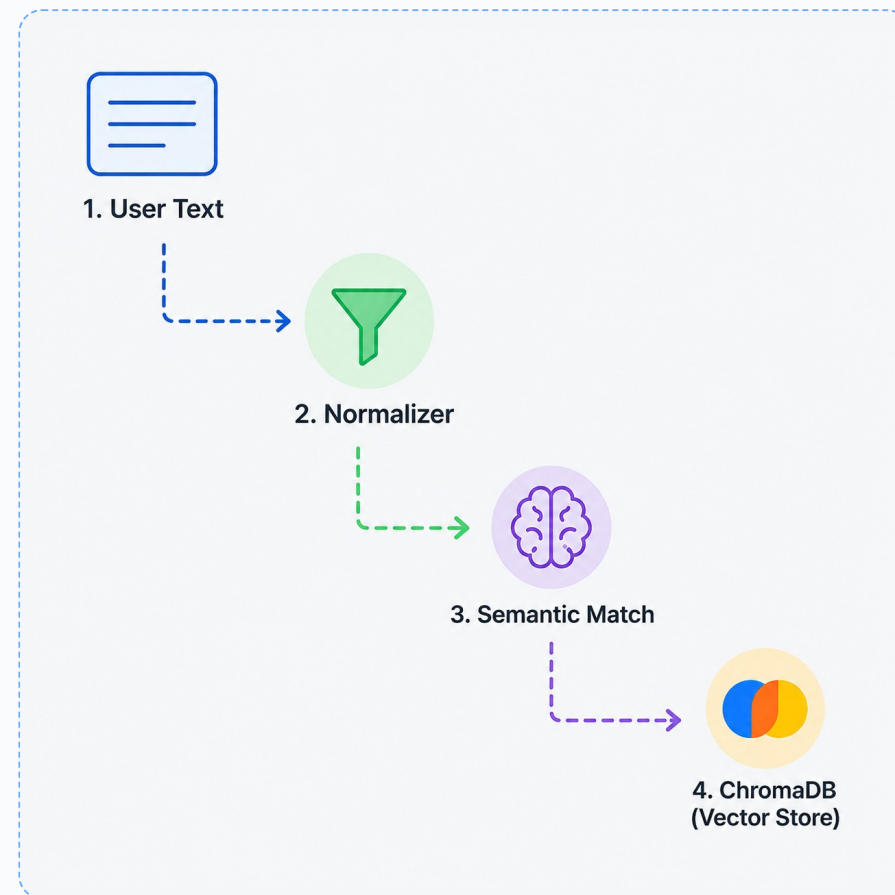
COMPONENT 02

Semantic Cache (ChromaDB)

Converts the query into dense vector embeddings. If a query has a 95% semantic match to a past query, it completely bypasses the LLM for immediate, reliable retrieval.

✓ 95% semantic-match threshold

✓ Bypasses LLM on hit



Context & Code Generation

Narrow the schema, prime the prompt, and let GPT-OSS 20B draft the SQL — with a 2-retry safety net.

01 PRE-PROCESSING

02 LLM GENERATION

03 SAFETY & EXECUTION

04 REPORTING



STEP 01

Schema Linker

Dynamically selects only the relevant table structures (out of **11** available) based on the query to prevent LLM hallucination and context-window overflow.



STEP 02

Prompt Builder

Injects the narrowed-down schema and validated “few-shot” examples to guide the LLM.



STEP 03

GPT-OSS 20B Engine

Takes the contextual prompt and generates the raw **PostgreSQL** statement. Features an automated **2-retry** recovery loop if syntax is slightly off.

Safety & Execution

Validate before you execute. Format before you return. Every query passes through AST validation.

- 01 PRE-PROCESSING
- ›
- 02 LLM GENERATION
- ›
- 03 SAFETY & EXECUTION
- ›
- 04 REPORTING



STEP 01

SQL Validator (SQLGlott)

Parses the generated SQL into an Abstract Syntax Tree (AST) to rigidly block any malicious intent (`DROP` , `DELETE` , SQL injections) before execution.

- ❌ DROP
- ❌ DELETE
- ❌ INJECTION



STEP 02

Query Executor

Runs the sanitized query against the `PostgreSQL` database as a read-only role (`querycraft_user`).



STEP 03

Report Generator

Takes the raw database rows and formats them into `JSON` for UI charts, or triggers direct byte-streams for downloads.

- CSV
- Excel
- PDF
- JSON

Security & Data Integrity

Four-Layer Defense System

01



Strict LLM Prompting

Forced instructions to generate `SELECT` queries exclusively. The model is constrained at the prompt level to never emit data-modifying SQL.

 LLM LAYER

02



AST Validation

`SQLGlot` completely prevents forbidden keywords and injection patterns. Every query is parsed into a syntax tree and rejected if it contains DDL or DML outside `SELECT`.

 PARSER LAYER

03



Read-Only Database Role

The system connects as `querycraft_user`, a locked-down role physically incapable of modifying the database — even if every other layer failed.

 DB LAYER

04



Hard Limits

Enforced **30-second** execution timeouts and hard caps at **10,000 rows** to prevent Denial-of-Service constraints.

 RUNTIME LAYER

Performance Optimizations

Where QueryCraft Got Fast

⚡ CACHE HIT

2s

Semantic Caching

Cached queries return in approximately 2 seconds. `ChromaDB` vector embeddings recognize semantically equivalent questions and short-circuit the LLM entirely.

🔍 NEW QUERY

15s

End-to-End Query Generation

New queries that bypass the cache take a maximum of 15 seconds end-to-end — from natural language input through schema linking, LLM generation, AST validation, to final result.

📄 EXPORTS

0 temp files

In-Memory Reporting

`CSV`, `Excel`, and `PDF` exports are handled entirely in memory as byte streams, keeping the server lean and avoiding temporary file clutter.

Tools & Frameworks

A modern, mostly-Python backend with a React frontend and an AI core built around GPT-OSS 20B + ChromaDB.

FRONTEND



React



Vite



TypeScript



Tailwind CSS



shadcn/ui



Recharts

BACKEND



Python



FastAPI



Uvicorn

AI & EMBEDDINGS



GPT-OSS 20B



sentence-transformers



BAAI/bge-large



ChromaDB

DATABASE & TOOLING



PostgreSQL



psycopg2



sqlglot



SQLite (audit logging)

EXPORT ENGINES



openpyxl



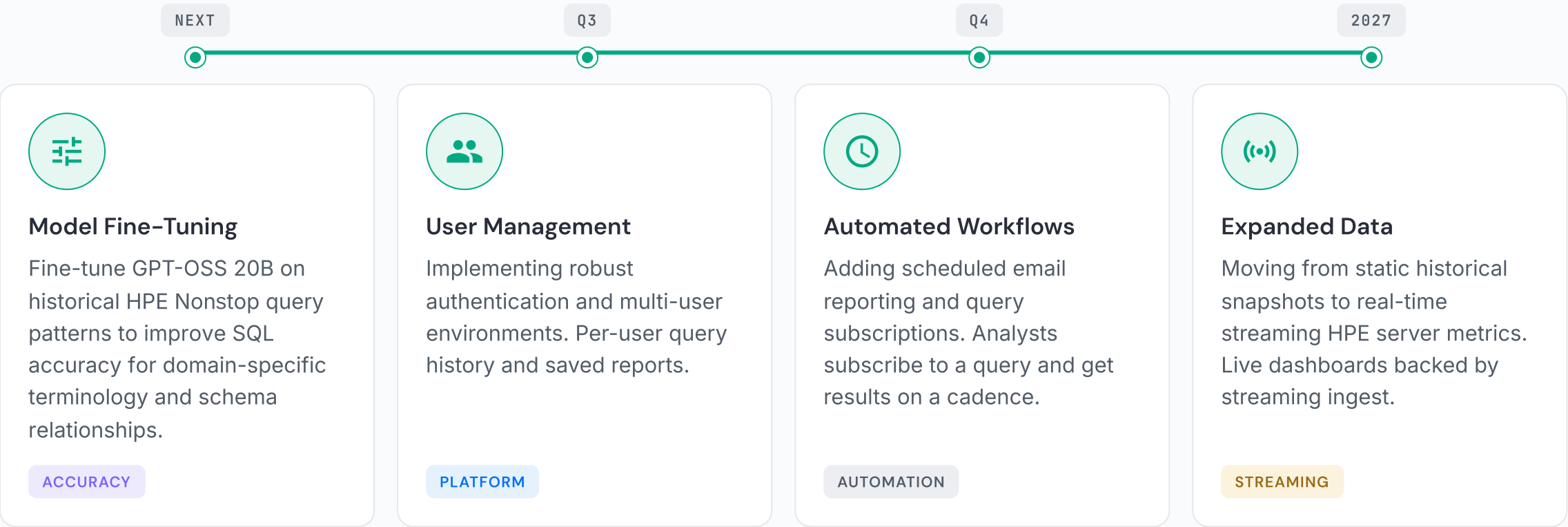
reportlab



weasyprint

Future Scope & Enhancements

Four planned directions — from offline AI to real-time streaming — building on the production-ready baseline.



Key Learnings

Three Lessons That Shaped QueryCraft

LESSON 01



Context is King

Feeding an LLM the entire database schema causes hallucinations. Dynamic Schema Linking — providing only what it needs — drastically improves SQL accuracy.

LESSON 02



Trust, but Verify

You can never implicitly trust LLM-generated code. AST validation (sqlglot) is an absolute necessity before touching a database.

LESSON 03



Data Types Matter

Microsecond mismatches across timestamp columns initially broke complex JOINS, teaching us the importance of explicitly prompting the LLM on how to align mismatched data (DATE_TRUNC).

Conclusion

Mission Accomplished. Baseline Set.



Mission Accomplished

QueryCraft successfully bridges the gap between non-technical analysts and complex database architectures. What used to require a SQL expert now requires only a sentence.



High Value Delivered

Transforms hours of manual SQL writing into seconds of plain English interaction. Analysts move from query mechanics to insight interpretation.



Production-Ready Baseline

Delivers a secure, highly optimized, and incredibly scalable foundation for AI-assisted data analytics. Ready to extend — not rebuild — for the next phase.

Thank You

